UCS645

PARALLEL & DISTRIBUTED COMPUTING

ASSIGNMENT 6:

Submitted by: Vansh Bhasin (102303270)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Part A – Device Query

1.1 Source Code

The following CUDA program queries all relevant GPU properties using the cudaDeviceProp structure from the CUDA Runtime API:

```
// Assignment 6 - Part A: Device Query
// Queries GPU properties using CUDA Runtime API

#include <stdio.h>
#include <cuda_runtime.h>

int main() {
    int deviceCount;
    cudaGetDeviceCount(&deviceCount);
    printf("Number of CUDA devices: %d\n\n", deviceCount);

    for (int dev = 0; dev < deviceCount; dev++) {
        cudaDeviceProp prop;
        cudaGetDeviceProperties(&prop, dev);

        printf("===== Device %d: %s =====\n", dev, prop.name);
        printf("\n-- Architecture & Compute Capability --\n");
        printf("  Compute Capability:            %d.%d\n", prop.major, prop.minor);

        printf("\n-- Thread / Block / Grid Dimensions --\n");
        printf("  Max Threads per Block:         %d\n", prop.
        printf("  Max Block Dimensions:          [%d x %d x %
              prop.maxThreadsDim[0], prop.maxThreadsDim[1], p
        printf("  Max Grid Dimensions:           [%d x %d x %
              prop.maxGridSize[0], prop.maxGridSize[1], prop.maxGridSize[2]);
        printf("  Warp Size:                     %d\n", prop.warpSize);
        printf("  Max Threads per MultiProcessor: %d\n", prop.maxThreadsPerMultiProcessor);
        printf("  Number of SMs:                 %d\n", prop.multiProcessorCount);

        printf("\n-- Memory --\n");
        printf("  Total Global Memory:           %.2f MB\n",
              prop.totalGlobalMem / (1024.0 * 1024.0));
        printf("  Total Constant Memory:         %zu bytes (%zu KB)\n",
              prop.totalConstMem, prop.totalConstMem / 1024);
        printf("  Shared Memory per Block:       %zu bytes (%zu KB)\n",
              prop.sharedMemPerBlock, prop.sharedMemPerBlock / 1024);
        printf("  Shared Memory per SM:          %zu bytes\n", prop.sharedMemPerMultiprocessor);
        printf("  L2 Cache Size:                 %d bytes\n", prop.l2CacheSize);
        printf("  Memory Bus Width:              %d bits\n", prop.memoryBusWidth);
        printf("  Memory Clock Rate:             %d kHz\n", prop.memoryClockRate);

        printf("\n-- Double Precision Support --\n");
        // Devices with compute capability >= 1.3 support double precision
        if (prop.major > 1 || (prop.major == 1 && prop.minor >= 3)) {
            printf("  Double Precision:              SUPPORTED\n");
        } else {
            printf("  Double Precision:              NOT SUPPORTED\n");
```

int cudaDeviceProp::maxGridSize[3]

Maximum size of each dimension of a grid

Generate Copilot summary

```
        } else {
            printf("  Double Precision:                NOT SUPPORTED\n");
        }

        printf("\n-- Other Properties --\n");
        printf("  Clock Rate:                      %.2f MHz\n", prop.clockRate / 1000.0);
        printf("  Registers per Block:             %d\n", prop.regsPerBlock);
        printf("  Concurrent Kernels:              %s\n",
               prop.concurrentKernels ? "Yes" : "No");
        printf("  ECC Enabled:                     %s\n",
               prop.ECCEnabled ? "Yes" : "No");
        printf("  Unified Addressing:              %s\n",
               prop.unifiedAddressing ? "Yes" : "No");
        printf("\n");
    }

    // Answer Q3: Max threads for 1D grid with maxGrid=65535, maxBlock=512
    long long maxGrid1D = 65535;
    long long maxBlock1D = 512;
    printf("=== Q3 Calculation ===\n");
    printf("Max 1D Grid Dim = %lld, Max 1D Block Dim = %lld\n", maxGrid1D, maxBlock1D);
    printf("Max Threads = %lld x %lld = %lld\n",
           maxGrid1D, maxBlock1D, maxGrid1D * maxBlock1D);

    return 0;
}
```

1.2 Compilation & Execution

nvcc device_query.cu -o device_query ./device_query

1.3 Questions & Answers

Q1. Architecture and Compute Capability:

Modern consumer GPUs (e.g., NVIDIA RTX 3060) use the Ampere architecture with Compute Capability 8.6. Enterprise/data-center GPUs like the A100 use Ampere with CC 8.0. Older GTX cards may use Turing (CC 7.5) or Pascal (CC 6.1). The compute capability determines which CUDA features are available, such as tensor cores (from 7.0+), hardware ray tracing (7.5+), and enhanced double-precision (from 6.0+).

Q2. Maximum Block Dimensions:

For most modern NVIDIA GPUs, the maximum block dimensions are [1024 x 1024 x 64] with the constraint that the total number of threads per block cannot exceed 1024. For example, a block of (32 x 32 x 1) = 1024 is valid, while (32 x 32 x 2) = 2048 exceeds the limit and would fail at runtime.

Q3. Maximum Threads (1D Grid: maxGrid=65535, maxBlock=512):

Maximum threads = Max Grid Dimension × Max Block Dimension = 65,535 × 512 = 33,553,920 threads (approximately 33.5 million threads).

Max Grid Dimension (1D)	**65,535**
Max Block Dimension (1D)	512
Max Total Threads	65,535 × 512 = 33,553,920

Q4. When might a programmer NOT launch the maximum threads?

A programmer might avoid launching the maximum threads when: • The problem size is small and using fewer threads avoids overhead from idle threads and wasted resources. • The kernel uses significant shared memory or registers per thread — launching maximum threads can cause register spilling to local (global) memory, hurting performance. • The algorithm requires inter-thread synchronization; fewer threads per block can reduce synchronization overhead. • Profiling shows diminishing returns — more threads do not always mean higher throughput due to memory bandwidth saturation.

Q5. What can limit launching maximum threads?

Several hardware and software constraints can limit the maximum threads: • Register file size: Each SM has a fixed number of registers. If a kernel uses many registers per thread, fewer threads can be scheduled simultaneously. • Shared memory capacity: If a kernel allocates large shared memory per block, fewer blocks can reside on an SM concurrently. • Maximum occupancy: Limited by the combination of threads per SM and blocks per SM hardware limits. • GPU memory (DRAM): Insufficient global memory to hold input/output data. • Kernel launch overhead: Very large grids introduce launch latency. • CUDA API limits: The maximum grid/block dimensions imposed by the hardware.

Q6. What is shared memory? How much is on your GPU?

Shared memory is a fast, programmer-managed cache located on-chip within each Streaming Multiprocessor (SM). It is shared among all threads in a block and has much lower latency than global memory (~5–10x faster). Threads in the same block use it to communicate and reuse data. Modern GPUs (e.g., Ampere architecture) provide 48 KB – 100 KB of configurable shared memory per SM (part of the L1/shared memory bank that can be configured up to 164 KB on A100).

Q7. What is global memory? How much is on your GPU?

Global memory is the main DRAM on the GPU. It is accessible by all threads across all blocks and has the largest capacity but the highest latency (~400–800 cycles). It is persistent for the duration of the CUDA application. Modern consumer GPUs like the RTX 3060 have 12 GB of GDDR6 global memory; data-center GPUs like the A100 have 40 GB or 80 GB of HBM2/HBM2e memory.

Q8. What is constant memory? How much is on your GPU?

Constant memory is a special read-only memory region in global DRAM that is cached in a dedicated constant cache on each SM. It is optimized for broadcast reads — when all threads in a warp read the same address, the access is served in a single cache hit. All CUDA devices provide 64 KB of constant memory regardless of architecture.

Q9. What does warp size signify?

A warp is the basic unit of execution on the GPU. The CUDA hardware executes threads in groups of 32 (the warp size) in lock-step using SIMT (Single Instruction, Multiple Threads). If threads within a warp diverge (e.g., due to an if-else), the GPU serializes the divergent paths, reducing efficiency. Understanding warp size is critical for writing efficient kernels. All NVIDIA GPUs have a warp size of 32.

Q10. Is double precision supported?

Double precision (float64) support depends on compute capability. GPUs with CC >= 1.3 support double precision. However, performance varies greatly: consumer GPUs (CC 6.1, 7.5, 8.6) perform double-precision at 1/32 the rate of single-precision (FP32:FP64 ratio of 32:1). Data-center GPUs like the A100 (CC 8.0) have FP32:FP64 ratio of 2:1. For most modern NVIDIA GPUs, double precision is supported but may be significantly slower than single precision on consumer hardware.

Part B – Array Sum (Parallel Reduction)

2.1 Algorithm Explanation

A parallel reduction is used to sum all elements efficiently. Instead of sequential addition (O(n)), the parallel approach uses a tree reduction:

- Each thread loads one element into shared memory.
- In each iteration, the active threads reduce the problem size by half.
- After $\log_2(n)$ iterations, thread 0 of each block holds the block's partial sum.
- Partial sums from all blocks are collected and summed on the host.

2.2 Source Code

```cpp
// Assignment 6 - Part B: Sum of Array Elements using CUDA
// Parallel reduction to compute sum of a floating-point array

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cuda_runtime.h>

#define ARRAY_SIZE 1024      // Must be a power of 2 for simple reduction
#define BLOCK_SIZE 256

// ------------------------------------------------------------
// CUDA Kernel: Parallel Reduction Sum
// Each block reduces its segment into a single partial sum stored
// in partialSums[blockIdx.x].
// ------------------------------------------------------------
__global__ void sumReductionKernel(float *input, float *partialSums, int n) {
    // Shared memory for this block
    __shared__ float sharedData[BLOCK_SIZE];

    int tid       = threadIdx.x;
    int globalIdx = blockIdx.x * blockDim.x + threadIdx.x;

    // Load element into shared memory (0 if out of bounds)
    sharedData[tid] = (globalIdx < n) ? input[globalIdx] : 0.0f;
    __syncthreads();

    // Reduction in shared memory (tree reduction)
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sharedData[tid] += sharedData[tid + stride];
        }
        __syncthreads();
    }

    // Thread 0 writes the block's partial sum
    if (tid == 0) {
        partialSums[blockIdx.x] = sharedData[0];
    }
}

// ------------------------------------------------------------
// Host helper: CPU reference sum for verification
// ------------------------------------------------------------
float cpuSum(float *arr, int n) {
    float sum = 0.0f;
    for (int i = 0; i < n; i++) sum += arr[i];
```

```
    for (int i = 0; i < n; i++) sum += arr[i];
    return sum;
}

int main() {
    int n = ARRAY_SIZE;
    size_t bytes = n * sizeof(float);

    // ── Step 1: Allocate and initialize host array ──────────────
    float *h_input = (float *)malloc(bytes);
    if (!h_input) { fprintf(stderr, "Host malloc failed\n"); return 1; }

    srand(42);
    for (int i = 0; i < n; i++) {
        h_input[i] = (float)(rand() % 100) / 10.0f;   // values in [0, 10)
    }
    printf("Array size: %d elements\n", n);
    printf("First 5 elements: %.2f %.2f %.2f %.2f %.2f ...\n",
           h_input[0], h_input[1], h_input[2], h_input[3], h_input[4]);

    // ── Step 2: Allocate device memory ──────────────────────────
    float *d_input, *d_partialSums;
    int numBlocks = (n + BLOCK_SIZE - 1) / BLOCK_SIZE;

    cudaMalloc((void **)&d_input,       bytes);
    cudaMalloc((void **)&d_partialSums, numBlocks * sizeof(float));

    // ── Step 3: Copy host memory to device ──────────────────────
    cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice);

    // ── Step 4: Initialize thread block and kernel grid dimensions
    dim3 blockDim(BLOCK_SIZE);
    dim3 gridDim(numBlocks);
    printf("\nKernel launch: <<<%d, %d>>>\n", numBlocks, BLOCK_SIZE);

    // ── Step 5: Invoke CUDA kernel ──────────────────────────────
    sumReductionKernel<<<gridDim, blockDim>>>(d_input, d_partialSums, n);
    cudaDeviceSynchronize();

    // Collect partial sums on host and add them
    float *h_partialSums = (float *)malloc(numBlocks * sizeof(float));
    cudaMemcpy(h_partialSums, d_partialSums,
               numBlocks * sizeof(float), cudaMemcpyDeviceToHost);

    float gpuSum = 0.0f;
    for (int i = 0; i < numBlocks; i++) gpuSum += h_partialSums[i];
```

```
92      for (int i = 0; i < numBlocks; i++) gpuSum += h_partialSums[i];
93
94      // ── Step 6: Free device memory ──────────────────────────────
95      cudaFree(d_input);
96      cudaFree(d_partialSums);
97
98      // ── Step 7: Verify against CPU reference ────────────────────
99      float cpu = cpuSum(h_input, n);
100     printf("\nCPU Sum  = %.4f\n", cpu);
101     printf("GPU Sum  = %.4f\n", gpuSum);
102     printf("Diff     = %.6f\n", fabsf(cpu - gpuSum));
103     printf("Result   : %s\n\n", fabsf(cpu - gpuSum) < 1e-2f ? "PASS" : "FAIL");
104
105     free(h_input);
106     free(h_partialSums);
107     return 0;
108 }
109
```

2.3 Key Design Decisions

Kernel Approach	**Tree-based reduction in shared memory**
Block Size	256 threads per block (tunable)
Shared Memory Usage	BLOCK_SIZE × sizeof(float) per block
Grid Size	ceil(N / BLOCK_SIZE) blocks
Synchronization	__syncthreads() ensures consistency between reduction steps
Boundary Handling	Threads beyond array size load 0 (identity for addition)
Final Accumulation	Partial sums reduced on host (or second kernel pass)
Verification	CPU sum compared against GPU sum for correctness

2.4 Compilation & Execution

nvcc array_sum.cu -o array_sum -lm ./array_sum

Part C – Matrix Addition

3.1 Source Code

```cpp
// Assignment 6 - Part C: Matrix Addition using CUDA
// Adds two large integer matrices element-wise on the GPU

#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define ROWS       1024
#define COLS       1024
#define BLOCK_DIM  16       // 16x16 = 256 threads per block

// ---------------------------------------------------------------
// CUDA Kernel: Matrix Addition
//
// Analysis for an N x M matrix:
//   Floating-point operations  : N * M  (one addition per element)
//   Global memory reads        : 2 * N * M  (read A[i][j] and B[i][j])
//   Global memory writes       : N * M  (write C[i][j])
// ---------------------------------------------------------------
__global__ void matAddKernel(int *A, int *B, int *C, int rows, int cols) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < rows && col < cols) {
        int idx = row * cols + col;
        C[idx] = A[idx] + B[idx];   // 1 addition (1 FLOP), 2 reads, 1 write
    }
}

// ---------------------------------------------------------------
// Host: initialise matrix with random ints in [0, 100)
// ---------------------------------------------------------------
void initMatrix(int *mat, int rows, int cols) {
    for (int i = 0; i < rows * cols; i++)
        mat[i] = rand() % 100;
}

// ---------------------------------------------------------------
// Host: CPU reference addition for verification
// ---------------------------------------------------------------
void cpuMatAdd(int *A, int *B, int *C, int rows, int cols) {
    for (int i = 0; i < rows * cols; i++)
        C[i] = A[i] + B[i];
}

// ---------------------------------------------------------------
// Host: compare two matrices
// ---------------------------------------------------------------
```

```
// ------------------------------------------------------------
// Host: compare two matrices
// ------------------------------------------------------------
int compareMatrices(int *ref, int *gpu, int rows, int cols) {
    for (int i = 0; i < rows * cols; i++)
        if (ref[i] != gpu[i]) return 0;
    return 1;
}

int main() {
    int rows = ROWS, cols = COLS;
    size_t bytes = rows * cols * sizeof(int);

    printf("Matrix size: %d x %d (%zu MB each)\n", rows, cols,
           bytes / (1024 * 1024));

    // ── Allocate host memory ──────────────────────────────
    int *h_A   = (int *)malloc(bytes);
    int *h_B   = (int *)malloc(bytes);
    int *h_C   = (int *)malloc(bytes);      // GPU result
    int *h_ref = (int *)malloc(bytes);      // CPU reference

    srand(123);
    initMatrix(h_A, rows, cols);
    initMatrix(h_B, rows, cols);

    // ── Allocate device memory ────────────────────────────
    int *d_A, *d_B, *d_C;
    cudaMalloc((void **)&d_A, bytes);
    cudaMalloc((void **)&d_B, bytes);
    cudaMalloc((void **)&d_C, bytes);

    // ── Copy host → device ────────────────────────────────
    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);

    // ── Configure grid and block dimensions ───────────────
    dim3 blockDim(BLOCK_DIM, BLOCK_DIM);   // 16x16 threads per block
    dim3 gridDim((cols + BLOCK_DIM - 1) / BLOCK_DIM,
                 (rows + BLOCK_DIM - 1) / BLOCK_DIM);

    printf("Block dim : (%d x %d)\n", BLOCK_DIM, BLOCK_DIM);
    printf("Grid  dim : (%d x %d)\n", gridDim.x, gridDim.y);
    printf("Total threads: %d\n\n", gridDim.x * gridDim.y * BLOCK_DIM * BLOCK_DIM);

    // ── Launch kernel ─────────────────────────────────────
    matAddKernel<<<gridDim, blockDim>>>(d_A, d_B, d_C, rows, cols);
```

```
    // ── Launch kernel ──────────────────────────────────────────
    matAddKernel<<<gridDim, blockDim>>>(d_A, d_B, d_C, rows, cols);
    cudaDeviceSynchronize();

    // ── Copy device → host ─────────────────────────────────────
    cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);

    // ── Verify ─────────────────────────────────────────────────
    cpuMatAdd(h_A, h_B, h_ref, rows, cols);
    int correct = compareMatrices(h_ref, h_C, rows, cols);
    printf("Verification: %s\n\n", correct ? "PASS" : "FAIL");

    // ── Operation count analysis ───────────────────────────────
    long long total = (long long)rows * cols;
    printf("=== Kernel Operation Analysis ===\n");
    printf("  Total elements           : %lld\n", total);
    printf("  Floating-point ops (FLOP): %lld  (1 add per element)\n", total);
    printf("  Global memory reads      : %lld  (2 reads per element: A + B)\n", 2 * total);
    printf("  Global memo  Generate Copilot summary rite per element: C)\n", total);
    printf("  Arithmetic intensity     : %.4f FLOP/byte\n",
           (double)total / (3.0 * total * sizeof(int)));

    // ── Free device memory ─────────────────────────────────────
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    free(h_A); free(h_B); free(h_C); free(h_ref);
    return 0;
}
```

3.2 Kernel Analysis

For an N × M matrix addition (N = M = 1024):

Matrix Size	**N × M = 1024 × 1024 = 1,048,576 elements**
Q1: Floating-Point Operations	1,048,576 additions (1 addition per element)
Q2: Global Memory Reads	2,097,152 reads (read A[i][j] and B[i][j] per element)
Q3: Global Memory Writes	1,048,576 writes (write C[i][j] per element)
Arithmetic Intensity	1 FLOP / (3 × 4 bytes) ≈ 0.083 FLOP/byte (memory-bound)
Block Configuration	16 × 16 = 256 threads per block
Grid Configuration	64 × 64 = 4,096 blocks
Total Threads Launched	64 × 64 × 16 × 16 = 1,048,576 threads

3.3 Detailed Analysis

Q1 – Floating-Point Operations per Kernel Call

Each thread performs exactly one integer addition: C[idx] = A[idx] + B[idx]. Since the kernel is launched with exactly N×M threads (one per matrix element), the total number of arithmetic operations is:

Total FLOPs = N × M = 1,024 × 1,024 = 1,048,576 operations

Q2 – Global Memory Reads

Each thread reads two values from global memory — one from matrix A and one from matrix B — to compute the result. Therefore:

Global Memory Reads = 2 × N × M = 2 × 1,048,576 = 2,097,152 reads (i.e., 2 × 1,024 × 1,024 × 4 bytes = 8 MB read from DRAM)

Q3 – Global Memory Writes

Each thread writes one value to the output matrix C. Therefore:

Global Memory Writes = N × M = 1,048,576 writes (i.e., 1,024 × 1,024 × 4 bytes = 4 MB written to DRAM)

Performance Observation (Memory-Bound Kernel)

Matrix addition has very low arithmetic intensity (~0.083 FLOP/byte). This means the kernel is heavily memory-bound — the GPU spends most of its time waiting for data from DRAM rather than doing compute. This is a classic example of a bandwidth-limited kernel. Optimizations like tiling with shared memory or fusing with other operations can help improve efficiency.

3.4 Compilation & Execution

nvcc matrix_add.cu -o matrix_add ./matrix_add

Summary of All Parts

The following table summarizes the key takeaways and file names for each part of the assignment:

Part	File	Concept	Key Learning
A	device_query.cu	GPU Device Properties	cudaDeviceProp, architecture, memory hierarchy
B	array_sum.cu	Parallel Reduction	Shared memory, tree reduction, block-level sync
C	matrix_add.cu	2D Parallel Computation	2D grid/block layout, memory-bound kernels, DRAM analysis

How to Compile All Files

 # Part A – Device Query nvcc device_query.cu -o device_query && ./device_query
 # Part B – Array Sum nvcc array_sum.cu -o array_sum -lm && ./array_sum # Part C – Matrix Addition nvcc matrix_add.cu -o matrix_add && ./matrix_add